SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Recycling Executed Branch Outcomes to Improve Branch Prediction

Neel Henry Mandal

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Recycling Executed Branch Outcomes to Improve Branch Prediction

Recycling ausgeführter Sprungergebnisse für verbesserte Sprungvorhersage

Author:	Neel Henry Mandal
Examiner:	Prof. Dr. Pramod Bhatotia
Supervisor:	Dr. David Schall
Submission Date:	20.02.2026

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 20.02.2026

Neel Henry Mandal

Acknowledgments

I would like to thank my supervisor, Dr. David Schall, for his incredible support and continuous guidance throughout this thesis. His expertise in microarchitecture and branch prediction has greatly impacted my personal development and interest in this field.

I would also like to thank Prof. Dr. Pramod Bhatotia for providing the opportunity to work at the Systems Research Group of the Technical University of Munich (TUM).

Furthermore, I would like to thank Jiyang Chen for introducing me to the chair during my Bachelor's Praktikum in the Computer Systems Lab.

Lastly, I would like to thank my girlfriend and family for their incredible support during this Thesis.

Abstract

Modern high-performance processors rely heavily on accurate branch prediction to support wide pipelines and large instruction windows. While state-of-the-art branch predictors such as TAGE with Statistical Corrector and Loop Predictor (TAGE-SC-L) achieve high average accuracy, a significant fraction of mispredictions remain, often caused by systematically Hard-to-Predict branches (branches that contribute to 90% of mispredictions) (HTP). These mispredictions lead to costly pipeline flushes, which result in wasted resources.

This thesis explores an alternative approach to branch prediction by recycling false-path executed branch outcomes. Instead of discarding resolved branch results after execution, the proposed mechanism stores and selectively reuses them to predict future branch occurrences. The design is implemented as an extension to the gem5 simulator and supports a base predictor, specifically TAGE-SC-L. Two optional control mechanisms, a per-branch training policy and a stride detection mechanism, are introduced to ensure that recycling is only applied to beneficial branches.

The evaluation uses both a controlled microbenchmark and a standardized benchmark suite including SPEC CPU and modern server workloads. Results show that branch outcome recycling significantly reduces branch mispredictions for repetitive and HTP branches, achieving an average misprediction reduction for a standard benchmark suite of 12.8% relative to TAGE-SC-L, while requiring only moderate additional storage overhead.

Overall, this work demonstrates that recycling committed branch outcomes is a viable and effective extension to contemporary branch predictors, offering improved accuracy under realistic storage constraints.

Contents

Acknowledgments **iv**

Abstract **v**

1 Introduction **1**
1.1 Motivation 1
1.2 Problem Statement 2
1.3 Thesis Goal and Research Questions 2
1.4 Contributions 3

2 Background **4**
2.1 State of the Art: TAGE-SC-L 4
2.2 Previous Work 5
2.2.1 Recycling Waste: Exploiting Wrong Path Execution to Improve Branch Prediction 5
2.3 The gem5 simulator system 6
2.3.1 Branch Prediction in gem5 6

3 System Overview - Design **7**
3.1 Overview 7
3.2 Recycling Mechanism 7
3.3 Branch Recycling Cache 8
3.3.1 Training 9
3.3.2 Stride Detection 10
3.4 Design Goals 10

4 Implementation **11**
4.1 Integration into gem5 11
4.2 Branch Recycling Cache Implementation 11
4.3 Training Mechanism Implementation 12
4.4 Stride Detection Implementation 12
4.5 Storage Requirements 13

5 Evaluation **14**
5.1 Experimental Setup 14
5.2 Methodology 15
5.3 Microbenchmark 16
5.4 Benchmark Test Suite 17
5.5 Parameter Sensitivity Study 17
5.5.1 Training Mechanism 17
5.5.2 Stride Detection 20
5.6 BRC Capacity Optimization 22
5.6.1 Bucket Count 22
5.6.2 FIFO queue Size 25
5.6.3 Combined 27
5.6.4 BRC Capacity Optimization conclusion 33
5.7 HTP-Branch Analysis 33

6 Summary and Conclusion **36**

7 Future Work **37**
7.1 Branch Recycling Cache (BRC) 37
7.2 Training Mechanism 37
7.3 Resource Usage 37
7.4 Selection of Base Predictor 38
7.5 Latency 38

8 AI Disclaimer **39**

Abbreviations **40**

List of Figures **41**

List of Tables **42**

Bibliography **44**

1 Introduction

1.1 Motivation

As Moore's law continues to slow down, performance improvements in modern processors increasingly rely on microarchitectural innovations rather than higher transistor counts. Current CPUs achieve higher performance primarily through wide, deep pipelines and large instruction windows that exploit instruction-level parallelism. These techniques place increasing pressure on the processor's frontend, which must continuously supply instructions to keep the backend busy.

One of the most critical components of the frontend is the branch predictor. Its task is to predict the outcome of control flow instructions before they are resolved, allowing the processor to speculatively fetch and execute instructions along the predicted execution path. Accurate branch prediction is essential to maintaining high pipeline utilization, as incorrect predictions trigger pipeline flushes and can result in hundreds of wasted instructions, leading to significant performance penalties. The cost of such mispredictions increases with deeper pipelines and larger instruction windows, making branch prediction increasingly important in modern and future processor designs [LT19].

State-of-the-art branch predictors, such as TAGE-SC-L, achieve very high average prediction accuracy by exploiting correlations in global branch history. TAGE-SC-L builds upon the TAgged GEometric history length (TAgged GEometric history length Predictor (TAGE)) predictor introduced in 2006 [SM06] and represents the current best performing design under realistic storage constraints. Despite this success, recent studies have shown that branch prediction is far from being a solved problem. Chit-Kwan Lin et al. show that a small number of systematically HTPbranches are responsible for a disproportionately large fraction of mispredictions, even with TAGE-SC-L [LT19]. These remaining mispredictions offer a substantial performance opportunity.

Simply increasing the size of existing branch predictors does not yield improved results. Conventional branch predictors rely on historical branch outcome information. The major flaw is predicting branches whose behavior does not correlate well with historical patterns. This observation motivates exploring new sources of information to support traditional branch prediction techniques.

1.2 Problem Statement

The main idea of recycling branch outcomes is to store the outcomes of squashed false-path branches with independent behavior. Although these outcomes are correct, they are generally discarded during pipeline recovery after a misprediction. A simple example of such a scenario is a loop that iterates over an array filled with random values between 1 and 10. During an iteration, the branch is taken if the array value is less than 5. Else, it is not. Since each iteration operates on a different array element, the branch outcomes are independent. For example, the CPU may speculatively execute iterations 1 through 10. If the branch in iteration 2 is mispredicted, the processor squashes all younger in-flight instructions, including iterations 3 through 10. However, the branch outcomes of these iterations may already have been computed correctly. Recycling allows these false path outcomes to be stored and recycled instead of being discarded. This raises the question of whether executed branch outcomes, which represent reliable execution behavior, can be recycled to improve future branch predictions. This thesis investigates the following problem:

Can branch prediction accuracy be improved by recycling executed branch outcomes in addition to conventional history-based prediction?

The central idea is to explore whether recently executed branch outcomes can be stored and selectively reused to predict future occurrences of the same branch, particularly for branches that are difficult to predict using traditional techniques.

1.3 Thesis Goal and Research Questions

The goal of this thesis is to evaluate the effectiveness and feasibility of recycling executed branch outcomes as an extension to the current state-of-the-art branch predictors. Specifically to answer the following research questions:

1. Does recycling executed branch outcomes reduce the number of branch mispredictions compared to a state-of-the-art baseline predictor?

2. Can executed outcomes be stored and reused with reasonable hardware and storage resources?

3. Does recycling executed outcomes reduce the impact of systematically HTP branches as identified by Chit-Kwan Lin et al. [LT19]?

By addressing these questions, this thesis aims to determine whether recycling executed outcomes is a viable extension of future branch predictors.

1.4 Contributions

This thesis makes the following contributions:

- The design and implementation of a new branch predictor Branch Recycling Predictor (BRP) in the gem5 simulator with a mechanism for recycling executed branch outcomes. The BRP is supported by a base predictor, in this case, the state-of-the-art predictor TAGE-SC-L (64 kB).
- A notification system that receives and stores executed branch outcomes from the processor pipeline.
- A self-managed (automatically evicts entries when capacity is reached) data structure for storing and retrieving executed outcomes under capped storage capacity: Branch Recycling Cache (BRC).
- A training policy that dynamically determines whether recycling is beneficial for a branch.
- A mechanism for detecting stride-based behavior in dynamic instructions, aiming at only using recycling during stable behavior, such as repetitive and tight loops.
- An evaluation of the proposed BRP using a custom microbenchmark with a single HTP branch, as well as a standard test suite.

2 Background

2.1 State of the Art: TAGE-SC-L

In this work, TAGE-SC-L is used as both the base branch predictor supporting the BRP and the main baseline for evaluation. The TAgged GEometric history length (TAGE) predictor was initially proposed by Seznec and Michaud in 2006 [SM06]. Since then, TAGE and its successors have consistently achieved the highest prediction accuracy in the Championship Branch Prediction (CBP) competitions, winning every competition except the most recent one. As a result, TAGE-based predictors are widely regarded as the state of the art in both academic research and industrial processor design.

Beyond academia, TAGE-based predictors are implemented in many modern high-performance CPUs. Vendors such as IBM, AMD, and Arm implement variants of TAGE in their processors, underlining their status as state-of-the-art predictors [LT19]. For this reason, TAGE-SC-L represents a strong and well-established baseline for evaluating new branch prediction mechanisms.

TAGE-SC-L extends the original TAGE, predictor by incorporating a perceptron-based Statistical Corrector and a loop predictor. The Statistical Corrector improves prediction accuracy by learning correlations that are difficult to capture with pure history-based matching, while the loop predictor targets loop branches with regular iteration counts, which are known to be challenging for TAGE alone. Together, these extensions address cases where the base TAGE predictor struggles, making TAGE-SC-L the current state-of-the-art conditional branch predictor [Sez16].

A TAGE predictor consists of a base bimodal predictor (T0) and multiple tagged predictor tables (T1-Tn), each indexed by a different global history length. These history lengths follow a geometric progression, allowing the predictor to capture both short-term and long-term correlations in branch behavior. Each tagged entry contains a prediction counter, a partial tag, and a usefulness bit used for replacement decisions. At prediction time, all components are accessed in parallel, and the prediction is taken from the matching tagged table with the longest history. If no tagged entry matches, the base predictor provides the prediction. When the confidence of the longest history prediction is low, an alternate prediction from a shorter history table may be selected to improve accuracy.

2.2 Previous Work

2.2.1 Recycling Waste: Exploiting Wrong Path Execution to Improve Branch Prediction

The paper "Recycling Waste: Exploiting Wrong Path Execution to Improve Branch Prediction," published in 2003, introduces a new mechanism to reduce branch mispredictions by recycling wrong path information that is usually discarded during squash. Modern processors prefetch and execute up to hundreds of instructions on the wrong path after a branch is mispredicted, and this work shows that those executions are not always useless. Because of control independence, branches executed on the wrong path often produce the same outcome as they would on the correct path. The authors propose using these wrong-path branch outcomes to improve future predictions rather than discarding them. The proposed mechanism, called branch recycling, stores branch outcomes from wrong-path executions in a small cache. When execution returns to the correct path, the predictor checks whether a stored wrong-path outcome matches the current branch instance. If so, the predictor can use that outcome to override the normal branch prediction. A lightweight predictor, called the Branch Recycling Predictor (BRP), decides when wrong-path outcomes are likely to be useful. This predictor uses global branch history and branch addresses and is implemented with simple 2-bit saturating counters. To store wrong path outcomes, the paper introduces a Branch Recycling Cache (BRC) that holds branch results in FIFO order. Each recycled outcome is used at most once and then discarded. The cache is sized to hold all branches in the instruction window, keeping hardware overhead low. The system is designed to work at the fetch stage so that corrections happen as early as possible, which is especially important for deep pipelines. The authors evaluate branch recycling using a detailed, cycle-accurate simulator based on a Pentium 4–like microarchitecture. They test 13 benchmarks from SPECint95, SPECint2000, multimedia, productivity, and web workloads. These benchmarks were chosen because they suffer from frequent branch mispredictions. The results show that branch recycling can reduce branch misprediction by up to 30%, and improve performance by 5% to 20%, depending on the pipeline depth. The benefits grow as pipelines become deeper, since deeper pipelines both suffer more from mispredictions and create more wrong-path executions to recycle. Realistic resource limitations, such as latency for wrong-path outcomes, are also evaluated. To implement this, the authors introduce a fetch throttling policy that briefly stalls fetches to wait for a useful false-path outcome. This policy significantly closes the gap between realistic hardware and an idealized oracle model. In conclusion, the paper shows that wasted wrong-path execution can be turned into a valuable resource for branch prediction. The proposed branch recycling mechanism is simple, effective, and becomes

increasingly useful as pipelines grow deeper, making it a promising approach for future high-performance processors.

2.3 The gem5 simulator system

The gem5 simulator is an open-source and community-supported simulator widely used in computer architecture research. It allows researchers to simulate modern computer systems at cycle-level accuracy, boot unmodified Linux operating systems, and run full applications. Further gem5 models execution and false-path behavior, providing higher accuracy than other simulators such as ChampSim. gem5 supports multiple ISAs, such as x86, ARM, and RISC-V, making it very useful for general micro- and architecture studies. The simulator is modular and highly configurable, allowing different hardware components such as CPUs, caches, and memory systems to be combined using Python scripts. gem5 is based on an event-driven simulation engine and separates functional execution from timing models. This separation enables features such as checkpointing, fast forwarding, and detailed timing analysis. In branch prediction research, the out-of-order (O3) CPU model is most commonly used because it provides higher microarchitectural accuracy than simpler models. This also applies to the use case in this research work. [Low+20]

2.3.1 Branch Prediction in gem5

gem5 provides explicit support for branch-prediction modeling and evaluation, making it suitable for control-flow-related research. Several branch predictors are implemented in gem5, and many of them were added or improved after the first release. Currently, gem5 includes industry-standard predictors such as TAGE, tournament predictors, and bi-mode predictors. Further, there is support for loop predictors and indirect branch predictors.

The branch prediction framework in gem5 is designed modularly, making it easy to extend existing predictors or add new ones. For example, the TAGE predictor can be combined with a loop predictor to better predict loops with constant iteration counts. Also, indirect branch predictors can be replaced with more complex schemes that require only simple configuration. Another important aspect is that branch predictors in gem5 are configurable in their size and history length. Researchers can adjust the number of tables, history register sizes, and other parameters, enabling them to study the trade-off between predictor complexity and accuracy. This makes gem5 very useful for evaluating branch mispredictions and overall performance under realistic workloads. [Low+20] This research uses the modularity of gem5 to develop a new predictor and benchmark it against the industry-standard TAGE-SC-L.

3 System Overview - Design

3.1 Overview

Figure 3.1 illustrates the high-level architecture of the proposed BRP. The BRP supports a conventional history-based branch predictor, such as TAGE-SC-L. The BRC stores previously executed branch outcomes and allows the BRP to reuse these outcomes for future occurrences of the same branch.

The base predictor is supported by the BRP. The BRP cannot function as a standalone predictor. Instead, the BRP acts as an optional override mechanism. During lookup, the base predictor is queried to obtain an initial outcome. In parallel, the BRC is indexed using the branch Program Counter (PC). If recycling is enabled and a valid recycled outcome exists, it may override the base prediction. Otherwise, the base predictor remains unchanged.

The BRP integrates directly into the processor pipeline's branch prediction framework. Specifically, the BRP is notified when conditional branches reach the commit stage of the out-of-order pipeline. After execution, the executed branch outcomes are correct and inserted into the BRC.

To ensure correctness, each recycled outcome is consumed exactly once and in program order. This preserves alignment between recycled entries and future occurrences of the branch.

The two optional control mechanisms, training and stride detection, dynamically determine whether recycling is beneficial for a given branch. These mechanisms aim to only recycle beneficial branch outcomes.

3.2 Recycling Mechanism

Branch recycling is implemented using per-branch FIFO queues that store recent branch outcomes. Each branch PC is associated with a dedicated queue wrapped in a bucket within the BRC.

A probe attached to the commit stage notifies the BRP whenever a conditional branch is committed. The BRP records the branch PC and its resolved outcome and inserts this information into the corresponding FIFO queue.

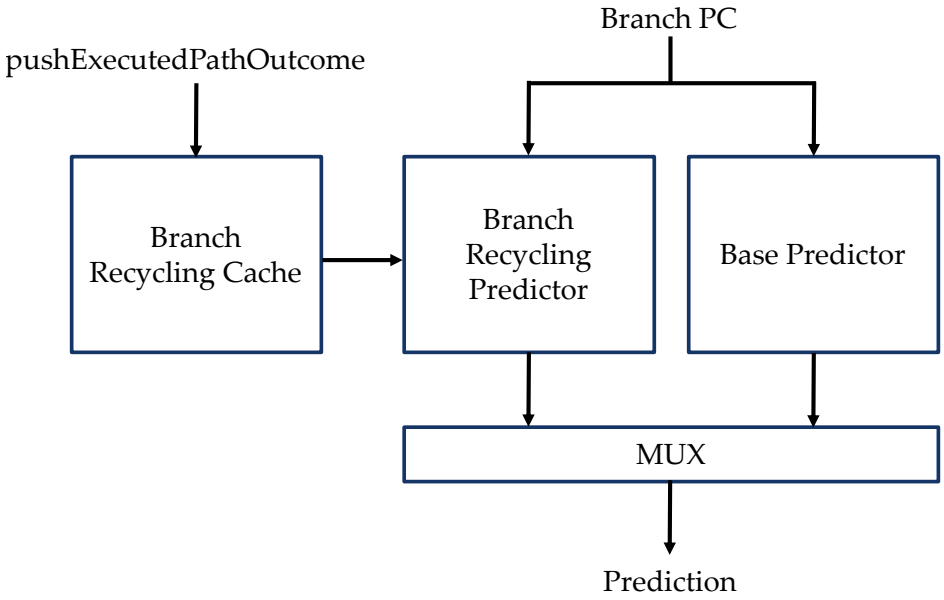

Figure 3.1: System overview of the proposed BRP

Restricting recycling to executed instructions ensures correctness. Storing outcomes before the commit stage would introduce incorrect data into the BRC. By inserting outcomes only at commit, the predictor aims to recycle outcomes only when the program executes correctly.

During lookup, both the BRC and the base predictor are queried. If recycling is enabled and a valid entry exists, the oldest entry in the FIFO queue is removed and used as the prediction. Each recycled outcome is consumed at most once to ensure proper alignment between stored outcomes and future executions.

If recycling is disabled or no valid entry exists, the base predictor is used instead. Even when the control mechanisms disable recycling, entries (if available) are still consumed to maintain correct alignment.

3.3 Branch Recycling Cache

The BRC (Example with training shown in 3.2 is implemented as a PC-indexed structure containing bounded FIFO queues. Each queue, wrapped by a bucket, stores recent branch outcomes for a specific branch.

Each bucket contains the following fields:

Branch Recycling Cache (BRC)					
TAG	LFU-Counter	Training Metric	Execs	Use Recycle	FIFO
0x01	5	5	3	TRUE	[0; 1; 0]
0x02	3	4	4	FALSE	[]
0x03	5	7	2	TRUE	[]
0x04	8	9	2	TRUE	[1; 1; 1]
0x05	0	0	0	FALSE	[]

pushExecutedOutcome

Figure 3.2: Example overview of the BRC for training

1. A tag identifying the branch PC.
2. A Least-Frequently Used (LFU) counter used for replacement decisions.
3. A bounded FIFO queue storing recent branch outcomes.

To cap the BRC, both the number of buckets and the size of each FIFO queue are capped. If a FIFO queue reaches capacity, the oldest entry is evicted. If the total number of buckets exceeds a fixed threshold, the least frequently used bucket is evicted. Age is used as a tiebreaker. These capacity limits ensure predictable storage requirements and enable efficient hardware implementation.

3.3.1 Training

When training is enabled, additional data is stored per bucket:

1. A training metric evaluating recycling accuracy relative to the base predictor.
2. An execution counter defining the training interval.
3. A control flag enabling or disabling the recycling during a training interval.

This information allows the BRP to dynamically determine whether recycling improves prediction accuracy. Training updates occur when a branch misprediction is detected. The actual outcome is compared against the predictions of both the base predictor and the BRP.

If recycling produces a correct prediction while the base predictor is incorrect, the training metric is incremented. If the base predictor is correct and recycling is incorrect, the metric is decremented.

After a fixed training interval, the metric is evaluated. Recycling is enabled when the metric is positive, indicating that it improves accuracy. Otherwise, it is disabled.

This mechanism allows the BRP to adapt to dynamic changes in program behavior.

3.3.2 Stride Detection

When stride detection is enabled, the BRP stores additional information related to dynamic memory access patterns:

1. A stride counter tracking stable stride behavior.
2. The last dynamic load address.
3. The most recent address offset.

If the two current consecutive offsets match, the stride counter is incremented. Otherwise, it is reset.

The BRP tracks the two most recent dynamic load addresses and computes the address offset between them. If the offset remains constant across the previous executions, the stride counter is incremented.

Once the stride counter exceeds a predefined threshold, the branch is classified as stable, and recycling is enabled. If the stride pattern breaks, the counter is reset, and recycling is disabled. This mechanism enables recycling only for repetitive and stable loop behavior.

3.4 Design Goals

The BRP was designed to evaluate the functionality of recycling executed branch outcomes. Besides the main target, implementing the correct functionality for recycling executed branch outcomes, the secondary target aims to achieve this with limited resource usage by capping the BRC. Logic cost of the BRP remains low but is not further evaluated.

4 Implementation

The following chapter describes the implementation of the proposed BRP in the gem5 simulator. The BRP is integrated as an extension to the existing branch prediction module of gem5 and operates alongside the base predictor.

4.1 Integration into gem5

The BRP is implemented as an extension of the existing branch predictor interface in gem5. The integration is performed within the out-of-order CPU model using the existing prediction, update, and commit interfaces. A probe is attached to the commit stage of the processor pipeline. This probe notifies the BRP whenever a conditional branch is committed. The probe provides the branch PC and the resolved branch outcome. The BRP maintains a BRC indexed by branch PC. Upon receiving a commit notification, the BRP inserts the resolved branch outcome into the corresponding bucket of the BRC. During prediction, the BRP is queried in parallel with the base predictor. If recycling is enabled and a valid entry exists, the BRP returns the recycled outcome. Otherwise, the base predictor result is used.

4.2 Branch Recycling Cache Implementation

The BRC is implemented as a bounded PC-indexed structure consisting of multiple buckets. Each bucket contains a fixed-size FIFO queue storing recent branch outcomes. Two capacity constraints are enforced:

- The maximum number of entries per FIFO queue.
- The maximum number of buckets in the BRC.

When a FIFO queue reaches its maximum capacity, the oldest entry is removed. When the total number of buckets exceeds the global capacity, the least frequently used bucket is evicted. The oldest access time is used as a tiebreaker.

This ensures bounded storage requirements and predictable memory usage.

4.3 Training Mechanism Implementation

The training mechanism is implemented using three per-branch variables stored in each bucket:

- `trainingMetric`
- `execs`
- `useRecycle`

These variables are updated during the BRP's update phase.

When a branch misprediction is detected, the actual outcome is compared against both the base predictor and the BRP. The training metric is incremented or decremented accordingly. If only the BRP predicted the correct outcome, the training metric is incremented. If only the base predictor predicted the correct outcome, the training metric is decremented. The training metric is not changed if both the BRP and base predictor predict the same outcome.

After a fixed number of executions, the training metric is evaluated. If the BRP performs at least as well as the base predictor, the `useRecycle` flag is enabled. Otherwise, recycling is disabled.

The counters are then reset for the next training interval.

4.4 Stride Detection Implementation

Stride detection is implemented by tracking dynamic load addresses associated with each branch.

Each bucket maintains the following fields:

- `lastDynAddr`
- `lastDynAddrOffset`
- `strideCounterDynAddr`

When a branch executes, the offset between consecutive dynamic addresses is computed. If the offset matches the previous offset, the stride counter is incremented. Otherwise, it is reset.

Recycling is enabled only when the stride counter exceeds a predefined threshold. To ensure there are no duplicate notifications in the pipe, the most recent sequence number is stored. If two consecutive sequence numbers match, the later notification is discarded.

4.5 Storage Requirements

The storage requirements for each bucket depend on the selected control mechanism.

Attribute	TAG	LFU-counter	trainingMetric	Execs	useRecycle	Entries
bits	$\log_2(M)$	16	3	3	1	N

Table 4.1: Storage requirements for a bucket with training and N entries in bits

Bucket size in bytes (training):

$$\text{Size} = \frac{\log_2(M) + 23 + N}{8}$$

Attribute	TAG	LFU	strideCounter	lastAddrOffset	lastAddr	Entries
bits	$\log_2(M)$	16	7	7	38	N

Table 4.2: Storage requirements for a bucket with stride detection and N entries in bits

Bucket size in bytes (stride detection):

$$\text{Size} = \frac{\log_2(M) + 68 + N}{8}$$

These storage requirements are used to evaluate the hardware overhead of the BRP, including the BRC.

5 Evaluation

5.1 Experimental Setup

All experiments were executed on `xavier`, a server operated by the Systems Research Group Chair at the TUM. The evaluation was performed using the gem5 simulator v 23.0.1.0 [1]. The experiments use the ARM64 architecture. Table 5.1 displays the gem5 configuration used during the evaluation.

Table 5.1: System configuration of gem5

Component	Configuration
Architecture	gem5 ARM ISA – full system mode
Processor Model	`SimpleProcessor` with out-of-order (`O3`) CPU model
Clock Frequency	3 GHz
Core Configuration	SPEC workloads: 1 core | Server workloads: 2 cores
Core Microarchitecture	6-way OoO, 512 ROB, 248/122 LQ/SQ
Memory	3 GiB dual-channel DDR4-2400
Branch Predictor	TAGE-SC-L and BRP with training and stride detection
BTB	16K entries, 8-way associative
Cache	32 KiB 8-way L1-I, 64 KiB 16-way L1-D, 2 MiB 16-way L2
Prefetcher	Instructions: FDIP + Tagged; Data: Stride

Branch prediction configuration The branch prediction unit (BPU) consists of a BTB, an indirect predictor (`ITTAGE`), and a conditional branch predictor. The conditional branch consists of two predictors, the BRP and the base predictor `TAGE-SC-L (64 kB)` as shown in Figure 3.1. The BRP can be configured in gem5 via variables to enable or disable recycling and its control mechanisms. The key parameters are:

- `enable_recycling`: enables recycling of executed branch outcomes.
- `enable_training`: enables per branch training mechanism.
- `enable_stride`: enables per branch stride detection.

[1] https://github.com/dhschall/gem5-fdp/tree/branch-recycling-cache

- `bucket_count` and `bucket_size`: control BRC capacity.
- `training_interval`: number of executions per training window.
- `stride_confidence_threshold`: threshold for enabling recycling during stride detection.

Statistics After the simulation completes, gem5 exports per-branch statistics to a CSV file (`stats.csv`) and overall statistics to a text file (`stats.txt`). These statistics are used to compute the evaluation metrics presented in Chapter 5. The key metrics are:

- Mispredictions per Kilo Instructions (MPKI).
- Mispredictions caused by the BRP.
- Mispredictions caused by faulty recycling.

5.2 Methodology

Figure 5.1: Methodology of Evaluation

The BRP is evaluated in five steps, as illustrated in Figure 5.1. First, the BRP is evaluated in an isolated scenario using a controlled microbenchmark. Second, it is evaluated on a benchmark suite with an uncapped BRC to establish baseline measurement values. Based on these baseline results, a parameter sensitivity analysis is conducted for the training and stride detection mechanisms. Next, the capacity of the BRC is optimized for both training and stride detection across three stages: first by stack/FIFO-queue size, second by bucket count, and lastly by optimizing both parameters. Lastly, a HTP branch analysis is conducted focusing on the precision of the BRP under both control mechanisms. Since gem5 simulations are deterministic under fixed configurations, each experiment was executed once. References to misprediction changes refer to changes in the MPKI, as execution and misprediction counts may vary across benchmarks.

5.3 Microbenchmark

To evaluate the hypothesis that recycling is beneficial for repetitive workloads with tight loops, a microbenchmark is implemented [2]. An array with 12,500 slots is filled with random values between 1 and 10. The loop then iterates over each entry in the array. If the value in the current field is less than 5, the branch is taken, else not. The workload is evaluated in the following predictor configurations:

1. `TAGE-SC-L (64 kB)`
2. BRP with `TAGE-SC-L (64 kB)` as the base predictor
3. BRP with `TAGE-SC-L (64 kB)` as the base predictor and training (threshold 5) enabled
4. BRP with `TAGE-SC-L (64 kB)` as the base predictor with stride detection (threshold 5) enabled

The BRC is intentionally not capped in size to avoid capacity-related effects and to analyze the impact and efficiency of recycling exclusively.

Table 5.2: Microbenchmark results in MPKI (lower better)

Entry	**TAGE-SC-L (64 kB)**	**Recycling**	**Training**	**Stride**
mispredictions due to DirectCond	7223	4318	4266	4437
number of instructions	155025	155025	155025	155025
MPKI	46.59	27.85	27.52	28.62

Table 5.3: Reduction in mispredictions relative to TAGE-SC-L (64 kB) in % (higher better)

Recycling	**Training**	**Stride**
40.22%	40.94%	38.57%

Due to the randomness of the microbenchmark, TAGE-SC-L performs as expected, with low accuracy and missing around half of the predictions. The recycling mechanism benefits significantly from the tight and repetitive loop, reducing mispredictions by

$$\frac{46.59 - 27.85}{46.59} \times 100 = 40.22\%$$

[2] https://github.com/dhschall/ubench/blob/branch-recycling/benchmarks/branch/branch_recycling.cc

without the use of control mechanisms. As the benchmark consists of a very homogeneous workload with a single dominant branch, the training and stride detection mechanisms are not required to detect beneficial branches and therefore do not strongly influence the results.

5.4 Benchmark Test Suite

Table 5.4: Overview of benchmarks with MPKI of TAGE-SC-L (64 kB)

Benchmark	**Language**	**Description**	**MPKI**
505.mcf[Löb17]	C	single-depot vehicle sched. in pub. transp.	24.91
523.xalancbmk[SPE17a]	C++	XSL Transformation	2.37
531.deepsjeng[SPE17b]	C++	AI - alpha-beta tree search & pattern recog.	9.85
541.leela[Pas17]	C++	AI - Monte Carlo simulation for GO	14.35
compression[Goo26]	C++	Snappy, ZSTD, Brotli, and Zlib	14.71
dacapo-luindex[Bla+06]	Java	Text indexing tool	2.97
dacapo-lusearch[Bla+06]	Java	Text search tool	6.54
dacapo-spring[Bla+06]	Java	Spring based application	6.39
mediawiki[Sch26]	PHP	Mediawiki page	9.32
nodeapp[Sch26]	JavaScript	Small shopping website	6.99
renaissance-chirper[PRL+19]	Scala/Java	Microblog. service using X Finagle	3.21
renaissance-http[PRL+19]	Scala/Java	Sends small Finagle req. to HTTP server	2.99
Average MPKI	**-**	**-**	**9.53**

To evaluate the behavior of the BRP under realistic workloads, the benchmark suite[3] shown in Table 5.4 is used, comprising SPEC CPU benchmarks and modern server-style applications. All benchmarks are executed using SimPoints. The BRP is evaluated against the `TAGE-SC-L (64 kB)` baseline under identical system configurations. The results reported in the following sections focus on misprediction rates and the effectiveness of the training and stride detection mechanisms.

5.5 Parameter Sensitivity Study

5.5.1 Training Mechanism

Threshold As previously described in section 4, the training cycle can be configured in gem5. The intervals 5, 10, 15, 20, 25, 50, 100, 250, 500, 1000, and 5000 are benchmarked using the recycling mechanism, with training enabled and an uncapped BRC to analyze

[3] https://github.com/dhschall/gem5-svr-bench

their effect. Table 5.6 and Figure 5.2 show the reduction of mispredictions relative to

Table 5.5: MPKI of Benchmark Suite for training interval (lower better)

Benchmark	**5**	**10**	**15**	**20**	**25**	**50**	**100**	**250**	**500**	**1000**	**5000**
505.mcf	26.32	26.47	26.39	26.52	26.37	26.41	26.41	26.91	27.08	27.23	27.27
523.xalancbmk	1.75	1.60	1.80	1.73	1.77	1.64	1.64	1.76	1.89	1.84	2.25
531.deepsjeng	7.18	7.26	7.35	7.27	7.29	7.37	7.37	7.47	7.60	7.64	8.20
541.leela	13.22	13.23	13.05	13.16	13.27	13.24	13.24	13.38	13.38	13.49	13.66
compression	15.23	15.29	15.22	15.28	15.25	15.38	15.38	15.74	15.53	15.87	16.32
dacapo-luindex	2.50	2.52	2.54	2.53	2.56	2.52	2.52	2.59	2.59	2.68	2.65
dacapo-lusearch	6.67	6.52	6.36	6.60	6.63	6.36	6.36	6.65	6.91	6.71	6.76
dacapo-spring	6.07	6.10	6.12	6.13	6.14	6.31	6.31	6.51	6.65	6.71	7.04
mediawiki	8.77	8.85	8.82	8.98	8.91	9.08	9.08	9.74	9.74	10.34	11.11
nodeapp	6.50	6.58	6.59	6.64	6.68	6.78	6.78	7.09	7.31	7.64	8.46
renaissance-chirper	2.77	2.77	2.91	2.89	2.87	3.06	3.06	3.35	3.48	3.65	3.73
renaissance-http	2.77	2.80	2.70	2.70	2.77	2.83	2.83	3.13	3.33	3.79	4.10
Average MPKI	8.31	8.33	8.32	8.37	8.38	8.42	8.42	8.69	8.79	8.97	9.30

TAGE-SC-L (higher better). Table 5.5 displays the MPKI of the conducted parameter evaluation (lower better). The results show that a low training interval of 5 delivers the overall best results. In relation to `TAGE-SC-L`, the recycling mechanism reduces mispredictions by an average of 12.78% with a training interval of 5.

Table 5.6: Reduction of mispredictions relative to TAGE-SC-L (64 kB) for training intervals in % (higher better)

Training Interval	**5**	**10**	**15**	**20**	**25**	**50**	**100**	**250**	**500**	**1000**	**5000**
505.mcf	-5.66%	-6.24%	-5.95%	-6.46%	-5.85%	-6.02%	-6.02%	-8.01%	-8.70%	-9.32%	-9.49%
523.xalancbmk	26.30%	32.45%	24.17%	27.07%	25.34%	30.90%	30.90%	25.73%	20.29%	22.57%	4.99%
531.deepsjeng	27.16%	26.32%	25.42%	26.18%	26.00%	25.18%	25.18%	24.16%	22.88%	22.45%	16.80%
541.leela	7.86%	7.82%	9.09%	8.27%	7.53%	7.73%	7.73%	6.75%	6.79%	6.00%	4.79%
compression	-3.52%	-3.93%	-3.45%	-3.88%	-3.66%	-4.52%	-4.52%	-6.95%	-5.56%	-7.89%	-10.93%
dacapo-luindex	15.65%	15.00%	14.50%	14.78%	13.82%	15.00%	15.00%	12.65%	12.79%	9.50%	10.74%
dacapo-lusearch	-4.47%	-2.09%	0.36%	-3.28%	-3.78%	0.32%	0.32%	-4.08%	-8.26%	-5.05%	-5.79%
dacapo-spring	17.36%	17.00%	16.68%	16.54%	16.39%	14.13%	14.13%	11.30%	9.51%	8.65%	4.09%
mediawiki	27.59%	26.90%	27.14%	25.82%	26.45%	24.98%	24.98%	19.58%	19.57%	14.63%	8.27%
nodeapp	29.07%	28.24%	28.10%	27.54%	27.08%	26.08%	26.08%	22.63%	20.21%	16.61%	7.76%
renaissance-chirper	53.85%	53.91%	51.48%	51.84%	52.12%	48.94%	48.94%	44.12%	42.05%	39.16%	37.81%
renaissance-http	34.00%	33.06%	35.47%	35.57%	33.95%	32.51%	32.51%	25.29%	20.46%	9.62%	2.24%
Average %	12.78%	12.57%	12.69%	12.18%	12.12%	11.69%	11.69%	8.78%	7.76%	5.92%	2.46%

Accuracy of the Training Mechanism To better understand the impact/effectiveness of the training mechanism, the MPKI′ are analyzed relative to the base predictor. The results in Table 5.6 show that training reduces overall mispredictions across most

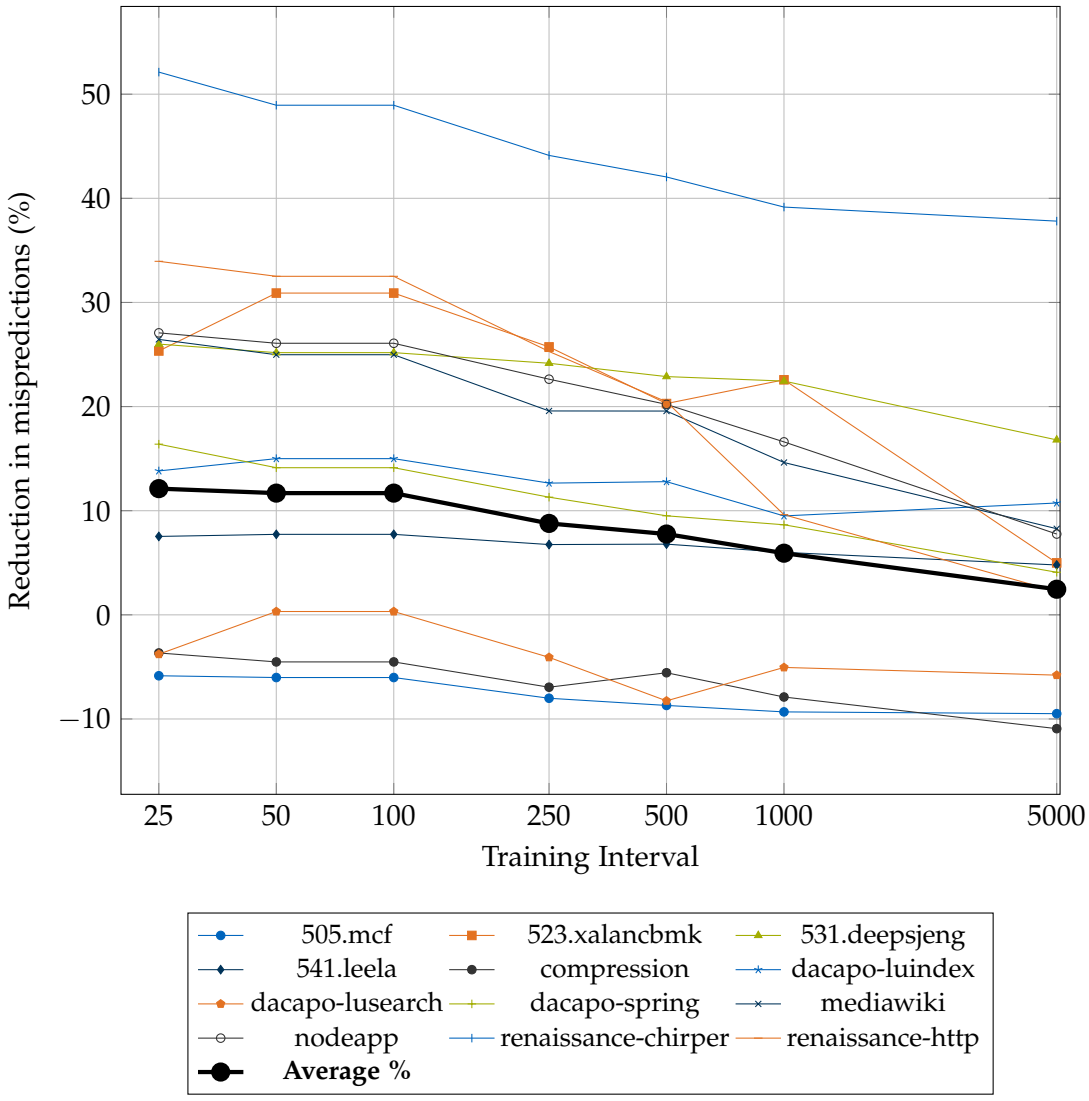

Figure 5.2: Reduction of mispredictions relative to TAGE-SC-L (64 kB) for training intervals (higher better)

benchmarks. While some workloads, such as `505.mcf` and `compression`, experience a slight increase in mispredictions, the majority of benchmarks benefit from recycling with training. In particular, server-style workloads such as `mediawiki`, `nodeapp`, and `renaissance-chirper` show a substantial reduction in mispredictions.

These results indicate that the simple training mechanism is effective at identifying branches that benefit from recycling while limiting negative impact on branches with irregular behavior. However, the mechanism does not fully eliminate faulty recycling and leaves room for improvement.

5.5.2 Stride Detection

Table 5.7: MPKI of benchmark suite for stride interval (lower better)

Benchmark	**5**	**25**	**50**	**100**	**250**
505.mcf	25.09	24.95	25.05	24.93	24.93
523.xalancbmk	2.28	2.48	2.24	2.28	2.28
531.deepsjeng	9.50	9.73	9.71	9.63	9.63
541.leela	14.40	14.17	14.23	14.33	14.33
compression	14.61	14.45	14.51	14.45	14.63
dacapo-luindex	3.15	3.40	3.29	3.34	3.34
dacapo-lusearch	6.59	6.40	6.61	6.78	6.78
dacapo-spring	7.21	7.02	6.95	7.29	7.02
mediawiki	11.46	11.88	11.55	11.72	11.91
nodeapp	8.55	8.96	8.69	8.68	8.96
renaissance-chirper	5.16	3.70	4.91	5.88	5.88
renaissance-http	4.17	4.17	4.33	4.18	4.18
Average MPKI	9.35	9.27	9.34	9.46	9.49

Threshold The stride detection threshold determines how many consecutive matching address offsets are required before recycling is enabled for a branch. To evaluate its impact, threshold values of 5, 25, 50, 100, and 250 are benchmarked using an uncapped BRC. Lower thresholds enable recycling earlier but may incorrectly mark patterns as strides, leading to inaccurate predictions. Higher thresholds improve stride detection but delay recycling activation and reduce potential benefits for short loops. Table 5.7 and Figure 5.3 show the reduction of mispredictions relative to TAGE-SC-L (higher better). Table 5.7 displays the MPKI of the conducted parameter evaluation (lower better). The results show that, on average, a stride interval of 25 delivers the best results. As recycling did not show beneficial results for the benchmark `505.mcf`, the highest

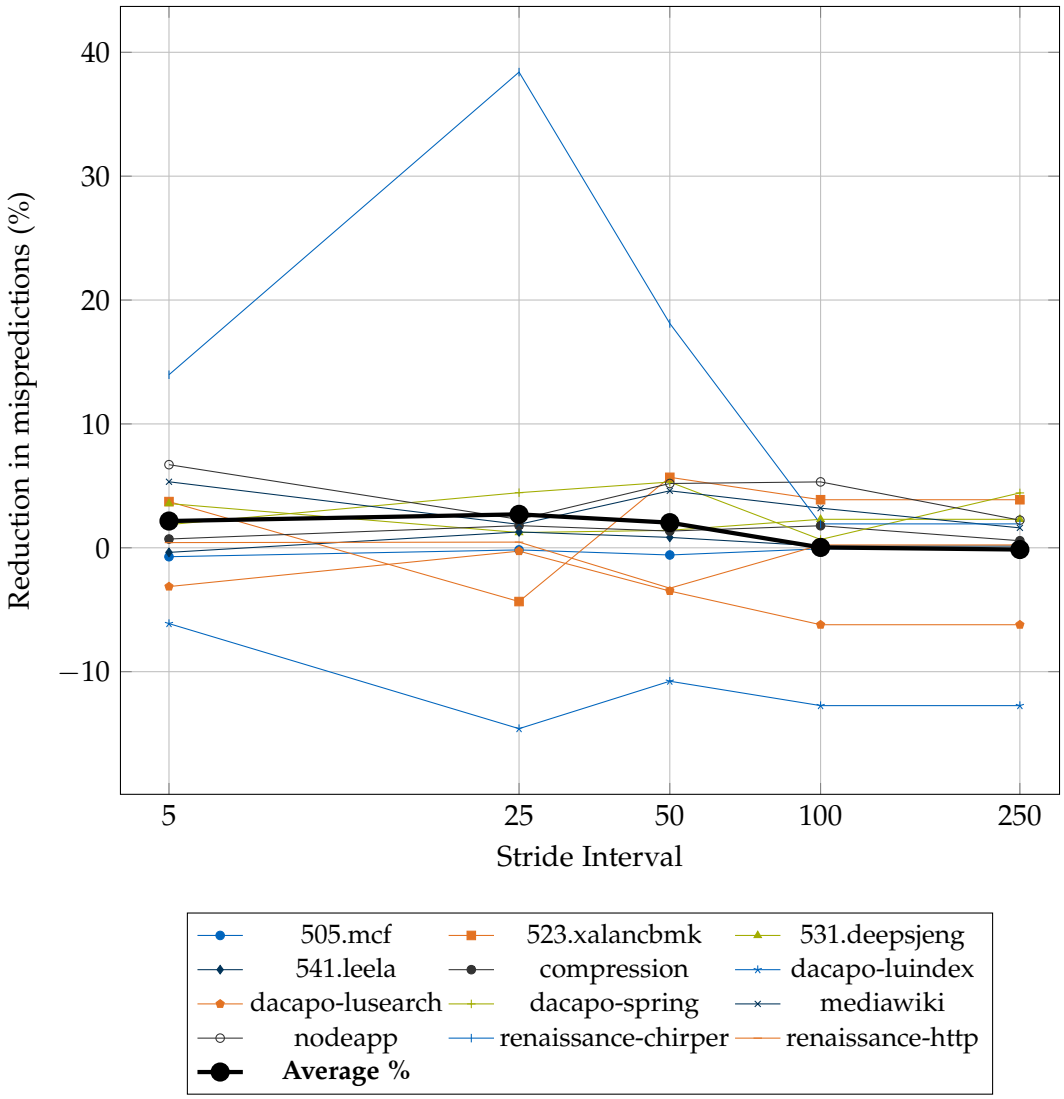

Figure 5.3: Reduction in mispredictions relative to TAGE-SC-L (64 kB) for stride intervals (higher better)

stride interval delivers the best results, because the base predictor is then more heavily utilized.

Accuracy of the Stride Detection As shown in Table 5.7 and Figure 5.3, stride detection provides low but consistent benefits for workloads with regular memory access behavior. Benchmarks such as `renaissance-chirper` and `dacapo` workloads benefit from early stride detection, while benchmarks with more irregular behavior show little to no improvement. Compared to the training mechanism, stride detection is more specialized and less adaptive. It performs best in workloads dominated by predictable loops but is less effective for applications with irregular workloads. As a result, stride detection alone is not sufficient as a universal control mechanism.

5.6 BRC Capacity Optimization

The BRC introduces two parameters that directly affect both storage overhead and prediction behavior: the number of buckets and the size of the FIFO queues. Increasing either parameter improves coverage of past branch outcomes but also increases storage cost. This section evaluates the impact of these two parameters. Both the number of buckets and the size of the individual FIFO queues in the BRC are capped independently at the following values: 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, and 8192. To analyze the effect of capping FIFO queue size and bucket count independently, either the bucket count or the FIFO queue size is capped while the other parameter remains uncapped. Finally, the optimal bucket count and FIFO queue size are evaluated in combination. The training and stride-detection mechanisms are assessed independently. The training mechanism operates with an interval of 5. The stride detection mechanism uses a threshold of 25.

5.6.1 Bucket Count

The bucket count determines how many branch PC′ can be recycled simultaneously.

Training

Table 5.8 displays the MPKI of the individual configurations and benchmarks. Table 5.9 shows the relative change in mispredictions to an uncapped BRC. A clear trend is visible: increasing the bucket count results in a lower MPKI. A few benchmarks stand out, `523.xalancbmk`, further reduces mispredictions by 9.69% relative to an uncapped BRC with a bucket count of 1024. Increasing the bucket count from 256 to 8192 reduces

Table 5.8: MPKI of benchmark suite for bucket count with training (lower better)

Bucket Count	**256**	**512**	**1024**	**2048**	**4096**	**8192**
505.mcf	26.32	26.32	25.67	25.67	25.67	25.67
523.xalancbmk	1.78	1.70	1.60	1.75	1.75	1.75
531.deepsjeng	7.97	7.36	7.18	7.18	7.18	7.18
541.leela	13.36	13.28	13.22	13.22	13.22	13.22
compression	15.15	15.23	15.23	15.23	15.23	15.23
dacapo-luindex	2.71	2.52	2.50	2.51	2.49	2.52
dacapo-lusearch	6.70	6.67	6.76	6.62	6.65	6.64
dacapo-spring	7.21	7.15	7.17	6.48	6.27	6.16
mediawiki	11.49	11.18	10.27	9.91	9.38	9.28
nodeapp	9.07	8.62	8.46	7.87	7.55	7.32
renaissance-chirper	5.32	5.31	3.26	3.37	2.99	3.12
renaissance-http	4.03	3.80	3.37	3.46	3.11	2.92
Average MPKI	9.26	9.09	8.78	8.66	8.51	8.47

Table 5.9: MPKI relative to an uncapped BRC for capped bucket count under training (lower better)

Buckets	**256**	**512**	**1024**	**2048**	**4096**	**8192**
505.mcf	0.00%	0.00%	-2.53%	-2.53%	-2.53%	-2.53%
523.xalancbmk	1.49%	-3.10%	-9.69%	-0.10%	-0.10%	-0.10%
531.deepsjeng	9.92%	2.51%	-0.06%	-0.06%	-0.06%	-0.06%
541.leela	1.06%	0.44%	0.02%	0.02%	0.02%	0.02%
compression	-0.52%	0.00%	0.00%	0.00%	0.00%	0.00%
dacapo-luindex	7.89%	0.96%	0.00%	0.55%	-0.46%	0.71%
dacapo-lusearch	0.42%	0.03%	1.26%	-0.68%	-0.28%	-0.42%
dacapo-spring	15.84%	15.12%	15.29%	6.35%	3.24%	1.44%
mediawiki	23.65%	21.52%	14.64%	11.54%	6.53%	5.48%
nodeapp	28.32%	24.58%	23.20%	17.42%	13.86%	11.25%
renaissance-chirper	47.90%	47.80%	15.08%	17.84%	7.51%	11.10%
renaissance-http	31.20%	27.10%	17.76%	19.95%	10.94%	5.27%
Average %	10.22%	8.60%	5.30%	4.03%	2.34%	1.88%

mispredictions from 10.22% to 1.88% relative to an uncapped BRC, highlighting the importance of providing enough buckets for the recycling with training mechanism.

Stride Detection

Table 5.10: MPKI of benchmark suite for bucket count with stride detection (lower better)

Bucket Count	**256**	**512**	**1024**	**2048**	**4096**	**8192**
505.mcf	24.95	24.95	24.95	24.95	24.95	24.95
523.xalancbmk	2.22	2.24	2.50	2.48	2.48	2.48
531.deepsjeng	10.00	9.51	9.73	9.73	9.73	9.73
541.leela	14.32	14.24	14.17	14.17	14.17	14.17
compression	14.48	14.45	14.45	14.45	14.45	14.45
dacapo-luindex	3.22	3.38	3.57	3.31	3.49	3.40
dacapo-lusearch	6.20	6.53	6.38	6.17	6.11	6.33
dacapo-spring	7.17	7.29	7.10	7.41	7.22	7.14
mediawiki	12.43	11.90	11.91	12.20	11.67	11.63
nodeapp	8.92	9.00	8.84	8.77	8.71	8.88
renaissance-chirper	5.51	3.62	6.87	5.42	5.44	6.45
renaissance-http	4.21	4.28	4.32	4.34	4.32	4.31
Average MPKI	9.47	9.28	9.56	9.45	9.39	9.49

Table 5.11: MPKI relative to an uncapped BRC for capped bucket count under stride detection (lower better)

Buckets	**256**	**512**	**1024**	**2048**	**4096**	**8192**
505.mcf	0.02%	0.02%	0.02%	0.02%	0.02%	0.02%
523.xalancbmk	-11.83%	-10.59%	0.82%	-0.19%	-0.19%	-0.19%
531.deepsjeng	2.74%	-2.30%	0.01%	0.01%	0.01%	0.01%
541.leela	1.07%	0.50%	-0.03%	-0.03%	-0.03%	-0.03%
compression	0.21%	-0.01%	-0.01%	-0.01%	-0.01%	-0.01%
dacapo-luindex	-5.44%	-0.61%	4.66%	-2.87%	2.44%	-0.01%
dacapo-lusearch	-3.24%	2.00%	-0.33%	-3.71%	-4.82%	-1.13%
dacapo-spring	2.14%	3.75%	1.08%	5.32%	2.73%	1.72%
mediawiki	4.43%	0.13%	0.28%	2.61%	-1.84%	-2.18%
nodeapp	-0.44%	0.46%	-1.41%	-2.18%	-2.83%	-0.91%
renaissance-chirper	32.85%	-2.22%	46.13%	31.75%	31.95%	42.65%
renaissance-http	0.95%	2.62%	3.40%	3.83%	3.48%	3.17%
Average %	2.06%	0.08%	3.02%	1.83%	1.25%	2.28%

Table 5.10 displays the MPKI of the individual configurations and benchmarks. Table

5.11 shows the relative change in mispredictions to an uncapped BRC. In contrast to training, stride detection is significantly less sensitive to bucket count. Even a configuration with 512 buckets increases mispredictions by only 0.08% relative to the uncapped design. No consistent trend is observable. This behavior can be explained by the purpose of stride detection. Since stride detection targets repetitive loop behavior, only a limited number of highly regular branches benefit from recycling. Therefore, even a moderate number of buckets is sufficient to cover the relevant working set.

5.6.2 FIFO queue Size

The FIFO queue size determines how many executed branch outcomes are stored per branch. In this analysis, the bucket count remains uncapped while the FIFO queue size is varied to isolate its effect on prediction accuracy.

Training

Table 5.12: MPKI of benchmark suite for FIFO queue size with training (lower better)

FIFO queue Size	**8**	**16**	**32**	**64**	**128**
505.mcf	26.27	26.21	26.35	26.30	26.28
523.xalancbmk	2.04	1.76	1.63	1.78	1.65
531.deepsjeng	7.37	7.23	7.28	7.24	7.20
541.leela	13.10	13.22	13.16	13.48	13.16
compression	15.22	15.16	15.19	15.26	15.21
dacapo-luindex	2.57	2.50	2.49	2.50	2.47
dacapo-lusearch	6.59	6.57	6.48	6.36	6.62
dacapo-spring	6.17	6.08	6.07	6.05	6.08
mediawiki	8.98	8.86	8.88	8.90	8.78
nodeapp	6.58	6.54	6.56	6.49	6.47
renaissance-chirper	4.22	2.85	3.73	2.87	2.85
renaissance-http	3.08	2.90	2.83	2.64	2.67
Average MPKI	8.52	8.32	8.39	8.32	8.29

Table 5.12 displays the MPKI of the individual configurations and benchmarks. Table 5.13 shows the relative change in mispredictions to an uncapped BRC. Increasing the FIFO queue size does not linearly reduce mispredictions as with the bucket count. A size of 16 already yields only a 0.14% increase relative to uncapped FIFO queues. Configuring the FIFO queues with 128 entries reduces mispredictions by 0.31% compared to uncapped FIFO queues.

Table 5.13: MPKI relative to an uncapped BRC for capped FIFO queues under training (lower better)

FIFO queue Size	**8**	**16**	**32**	**64**	**128**
505.mcf	-0.20%	-0.42%	0.12%	-0.06%	-0.16%
523.xalancbmk	14.24%	0.82%	-7.22%	1.72%	-5.78%
531.deepsjeng	2.58%	0.64%	1.34%	0.83%	0.33%
541.leela	-0.93%	-0.02%	-0.47%	1.91%	-0.48%
compression	-0.09%	-0.45%	-0.27%	0.16%	-0.12%
dacapo-luindex	2.61%	0.00%	-0.41%	-0.18%	-1.21%
dacapo-lusearch	-1.15%	-1.48%	-2.88%	-4.95%	-0.71%
dacapo-spring	1.55%	0.22%	-0.08%	-0.33%	0.11%
mediawiki	2.38%	1.05%	1.24%	1.49%	0.10%
nodeapp	1.26%	0.54%	0.98%	-0.14%	-0.40%
renaissance-chirper	34.38%	2.84%	25.65%	3.50%	2.73%
renaissance-http	10.06%	4.57%	2.20%	-5.12%	-3.78%
Average %	2.38%	0.14%	0.89%	0.11%	-0.31%

Stride Detection

Table 5.14: MPKI of benchmark suite for FIFO queue size with stride detection (lower better)

FIFO queue Size	**8**	**16**	**32**	**64**	**128**
505.mcf	24.97	24.96	24.96	25.02	25.20
523.xalancbmk	2.24	2.21	2.31	2.57	2.28
531.deepsjeng	9.38	9.72	9.50	9.67	9.87
541.leela	14.22	14.17	14.25	14.30	14.18
compression	14.68	14.84	14.68	14.68	14.83
dacapo-luindex	3.28	3.41	3.45	3.32	3.18
dacapo-lusearch	6.69	6.09	6.29	6.49	6.30
dacapo-spring	7.27	7.31	7.10	7.05	7.15
mediawiki	11.95	11.72	11.88	11.75	12.13
nodeapp	8.84	8.94	8.61	8.55	8.75
renaissance-chirper	4.15	5.39	7.06	6.57	9.11
renaissance-http	4.32	4.21	4.23	4.33	4.23
Average MPKI	9.33	9.42	9.53	9.53	9.77

Table 5.14 displays the MPKI of the individual configurations and benchmarks. Table 5.15 shows the relative change in mispredictions to an uncapped BRC. Compared to previous trends, increasing FIFO queue size increases the number of mispredictions, indicating that only very recent executed outcomes are beneficial for recycling under

Table 5.15: MPKI relative to an uncapped BRC for capped FIFO queues under stride detection (lower better)

FIFO queue Size	**8**	**16**	**32**	**64**	**128**
505.mcf	0.09%	0.42%	0.06%	0.28%	0.97%
523.xalancbmk	-10.78%	-12.28%	-7.57%	3.60%	-8.99%
531.deepsjeng	-3.76%	-0.11%	-2.44%	-0.60%	1.44%
541.leela	0.36%	0.02%	0.53%	0.90%	0.06%
compression	1.54%	2.63%	1.58%	1.55%	2.58%
dacapo-luindex	-3.51%	0.33%	1.36%	-2.51%	-6.83%
dacapo-lusearch	4.30%	-5.05%	-1.74%	1.32%	-1.60%
dacapo-spring	3.49%	3.93%	1.10%	0.44%	1.83%
mediawiki	0.55%	-1.34%	-0.03%	-1.07%	2.06%
nodeapp	-1.33%	-0.21%	-4.09%	-4.80%	-2.35%
renaissance-chirper	10.81%	31.39%	47.56%	43.69%	59.39%
renaissance-http	3.51%	0.89%	1.40%	3.78%	1.44%
Average %	0.61%	1.56%	2.62%	2.62%	5.03%

stride detection.

Conclusion

Overall, bucket count is identified as the dominant parameter for recycling with training. In contrast, there is no clear trend visible for the bucket count under stride detection. Training slightly benefits from an increase in FIFO-queue size, while stride detection suffers from an increase in FIFO-queue size.

5.6.3 Combined

After independently evaluating the impact of bucket count and FIFO queue size, the two parameters are jointly evaluated to analyze their combined effect on prediction accuracy under constrained storage for both training and stride detection.

Training

Table 5.16 presents the MPKI values for different combinations of bucket count and FIFO queue size when training is enabled with a training threshold of 5.

The results show a clear and consistent trend. Increasing the bucket count reduces mispredictions across all evaluated FIFO queue sizes. On the other hand, increasing the FIFO queue size does not have the same effect. Once a sufficient number of buckets is allocated, the reduction in misprediction between FIFO queue sizes diminishes.

Table 5.16: MPKI for training with capped cache (lower better)

Buckets / FIFO queue Size	8	16	32	64	128
128	9.43	9.55	9.22	9.44	9.51
256	9.26	9.13	9.12	9.43	9.34
512	9.20	8.91	9.14	8.93	8.90
1024	9.15	8.82	8.79	8.82	8.77
2048	8.82	8.64	8.67	8.78	8.61
4096	8.72	8.48	8.52	8.55	8.48
8192	8.60	8.44	8.44	8.43	8.41

Table 5.17: MPKI reduction relative to training with uncapped cache in % (lower better)

Buckets / FIFO queue Size	8	16	32	64	128
128	11.98%	13.07%	9.94%	12.08%	12.68%
256	10.37%	9.02%	9.01%	11.98%	11.13%
512	9.76%	6.80%	9.19%	6.98%	6.75%
1024	9.28%	5.89%	5.50%	5.84%	5.31%
2048	5.83%	3.90%	4.21%	5.39%	3.57%
4096	4.77%	2.15%	2.51%	2.93%	2.06%
8192	3.49%	1.59%	1.61%	1.51%	1.32%

Table 5.18: Storage requirements of BRC for training in kB

Buckets / FIFO queue Size	8	16	32	64	128
128	0.61	0.73	0.98	1.48	2.48
256	1.25	1.50	2.00	3.00	5.00
512	2.56	3.06	4.06	6.06	10.06
1024	5.25	6.25	8.25	12.25	20.25
2048	10.75	12.75	16.75	24.75	40.75
4096	22.00	26.00	34.00	50.00	82.00
8192	45.00	53.00	69.00	101.00	165.00

Table 5.19: MPKI reduction relative to TAGE-SC-L for recycling with training in % (higher better)

Buckets / FIFO Size	8	16	32	64	128
128	1.02%	-0.22%	3.26%	0.91%	0.22%
256	2.79%	4.24%	4.25%	1.02%	1.97%
512	3.46%	6.52%	4.06%	6.34%	6.57%
1024	3.97%	7.43%	7.81%	7.48%	7.99%
2048	7.49%	9.34%	9.05%	7.91%	9.65%
4096	8.52%	10.97%	10.63%	10.25%	11.05%
8192	9.73%	11.47%	11.45%	11.54%	11.71%

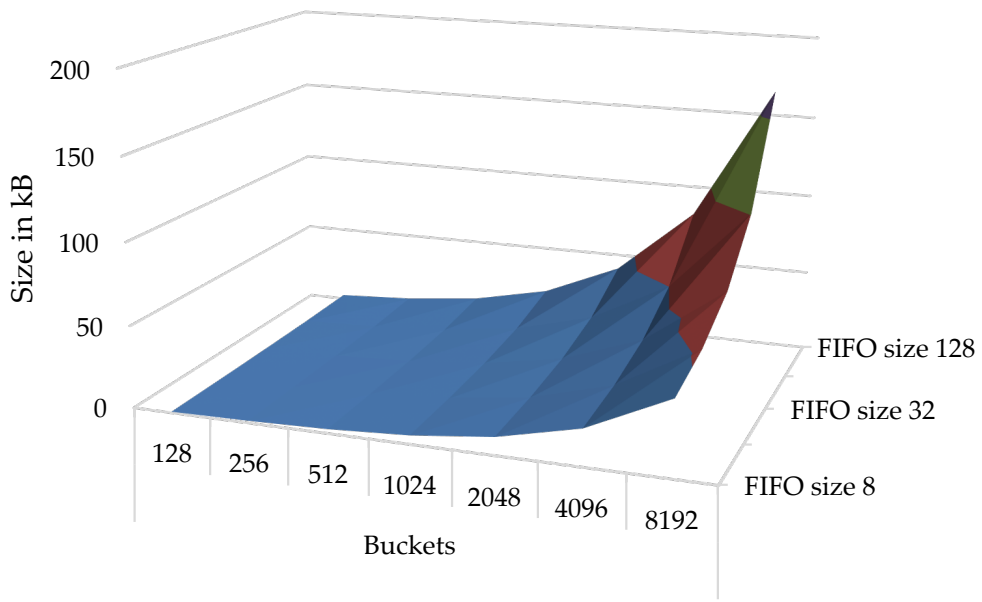

Figure 5.4: Storage requirements of BRC by bucket count and FIFO size in kB for training

For example, with a bucket count of 8192, the FIFO queue sizes 16 and 128 deliver mispredictions of 1.59% vs. 1.32%.

Across all configurations, the best-performing configuration is 8192 buckets with a FIFO queue size of 128, yielding an average MPKI of 8.41. However, several smaller configurations offer very similar accuracy at significantly lower storage costs. The best balance between overall storage size and performance is achieved by a configuration with 4096 buckets and a FIFO queue size of 16, yielding an average MPKI of 8.48, a marginal increase of 2.12% over the uncapped baseline.

The storage needs for the BRC of the evaluated configurations are presented in Table 5.18 and visualized in Figure 5.4. Mispredictions relative to TAGE-SC-L by bucket count and FIFO queue size are visualized in Figure 5.5 and presented in Table 5.19. Storage grows linearly with both the number of buckets and the FIFO queue size. For a fixed FIFO queue size, doubling the bucket count approximately doubles the overall storage consumption. Similarly, increasing the FIFO queue size results in a linear increase in storage per bucket. For example, a configuration with 4096 buckets and a FIFO queue size of 16 requires approximately 26 `kB` of storage, whereas increasing the FIFO queue size to 128 raises the requirement to 82 `kB`. The best-performing configuration with

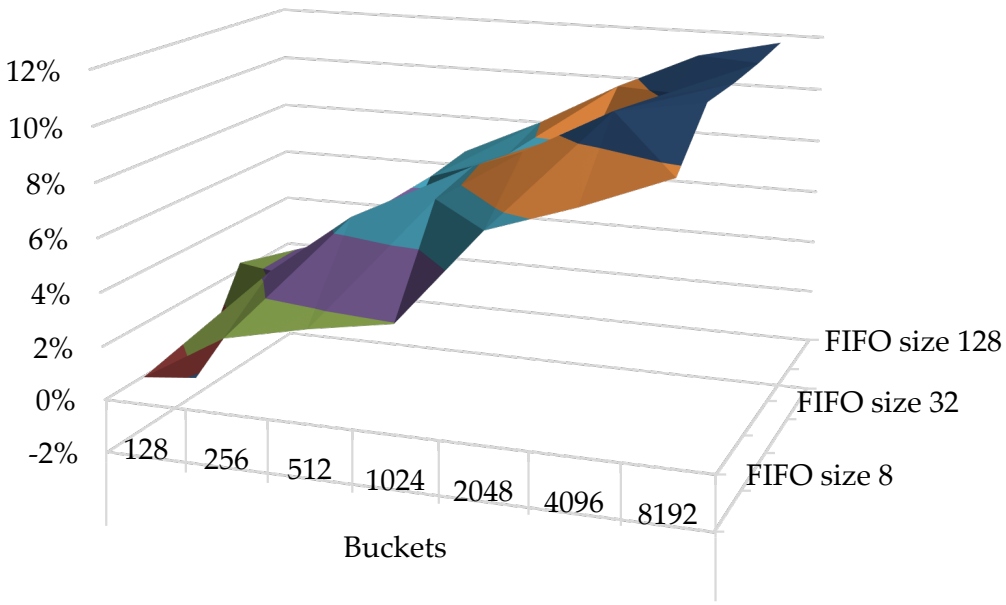

Figure 5.5: MPKI of recycling with training mechanism relative to TAGE-SC-L in % (higher better)

8192 buckets and FIFO queue size of 128 requires 165 kB. The configuration with 4096 buckets and FIFO queue size of 16, requires only 26 kB. A reduction of factor 6.35 while only increasing mispredictions by 2.04% vs. 1.32% relative to an uncapped BRC. These results indicate that bucket count is the primary factor for prediction accuracy, while FIFO queue size fine-tunes performance once sufficient bucket capacity is available. This makes joint optimization of both parameters an effective way to balance accuracy and storage overhead.

Stride Detection

Table 5.20 presents the MPKI values for different combinations of bucket count and FIFO queue size when stride detection is enabled. Table 5.21 presents the relative change of mispredictions compared to an uncapped BRC in %. In contrast to the BRP with training, the results show significantly less impact on storage capacity. Across nearly all configurations, the average MPKI remains close to the uncapped stride baseline of 9.49 ranging from .

Unlike training, increasing the number of buckets does not consistently improve prediction accuracy. While bucket counts such as 512 or 1024 occasionally deliver

Table 5.20: MPKI for stride detection with capped cache (lower better)

Buckets / FIFO queue Size	**8**	**16**	**32**	**64**	**128**
128	9.36	9.52	9.55	9.48	9.53
256	9.77	9.65	9.43	9.41	9.45
512	9.63	9.48	9.26	9.45	9.59
1024	9.44	9.29	9.54	9.51	9.44
2048	9.40	9.53	9.53	9.53	9.53
4096	9.52	9.45	9.30	9.50	9.43
8192	9.43	9.45	9.52	9.69	9.44

Table 5.21: MPKI reduction relative to stride detection with uncapped cache in % (lower better)

Buckets / FIFO queue Size	**8**	**16**	**32**	**64**	**128**
128	0.17%	1.90%	2.25%	1.45%	2.01%
256	4.36%	3.20%	1.01%	0.71%	1.22%
512	3.06%	1.49%	-0.81%	1.13%	2.57%
1024	1.02%	-0.50%	2.14%	1.75%	1.02%
2048	0.68%	1.98%	1.96%	1.95%	2.01%
4096	1.88%	1.14%	-0.44%	1.67%	0.94%
8192	0.95%	1.17%	1.89%	3.61%	1.03%

Table 5.22: Storage requirements of BRC for stride detection in kB

Buckets / FIFO queue Size	**8**	**16**	**32**	**64**	**128**
128	1.47	1.59	1.84	2.34	3.34
256	2.97	3.22	3.72	4.72	6.72
512	6.00	6.50	7.50	9.50	13.50
1024	12.13	13.13	15.13	19.13	27.13
2048	24.50	26.50	30.50	38.50	54.50
4096	49.50	53.50	61.50	77.50	109.50
8192	100.00	108.00	124.00	156.00	220.00

Table 5.23: MPKI reduction relative to TAGE-SC-L for recycling with stride detection in % (higher better)

Buckets / FIFO Size	**8**	**16**	**32**	**64**	**128**
128	1.83%	0.10%	-0.26%	0.55%	-0.01%
256	-2.47%	-1.25%	1.00%	1.29%	0.79%
512	-1.10%	0.51%	2.78%	0.88%	-0.59%
1024	0.99%	2.48%	-0.15%	0.25%	0.99%
2048	1.32%	0.02%	0.04%	0.04%	-0.02%
4096	0.11%	0.87%	2.43%	0.33%	1.07%
8192	1.06%	0.83%	0.11%	-1.68%	0.97%

slightly improved results (e.g., 9.26 MPKI for 512 buckets with FIFO queue size 32), the differences remain marginal. Even with a large configuration of 8192 buckets, there is no systematic improvement in accuracy.

Similarly, increasing FIFO queue size has only a limited influence. Small FIFO queue sizes, such as 8 or 16 entries, often perform comparably to larger configurations. This indicates that stride-based recycling typically benefits from only a small number of recently executed branch outcomes.

The storage requirements for stride detection are shown in Table 5.22 and Figure 5.6. Mispredictions relative to TAGE-SC-L by bucket count and FIFO queue size are visualized in Figure 5.7 and presented in Table 5.23.Storage scales linearly with both the bucket count and the FIFO queue size. Since large-scale configurations, such as 8192 buckets with a FIFO queue size of 128 (220 kB), only achieve minimal accuracy gains over mid-scale configurations, such as 4096 Buckets with a FIFO queue size of 16 (26 kB), the cost-performance ratio is not met.

The best balance between accuracy and storage overhead is achieved by moderate configurations such as:

- 512 buckets with FIFO queue size 32 (approx. 6.5 kB),
- or 1024 buckets with FIFO queue size 16 (approx. 11.1 kB).

These configurations deliver accuracy close to the uncapped stride baseline while keeping storage consumption relatively small.

Overall, the combined evaluation confirms that stride detection is less dependent on a large BRC compared to the training mechanism. Since stride detection targets a specific class of repetitive branches, it requires only limited history depth and bucket capacity. As a result, stride-based recycling can be implemented with modest hardware resources without significant loss in accuracy.

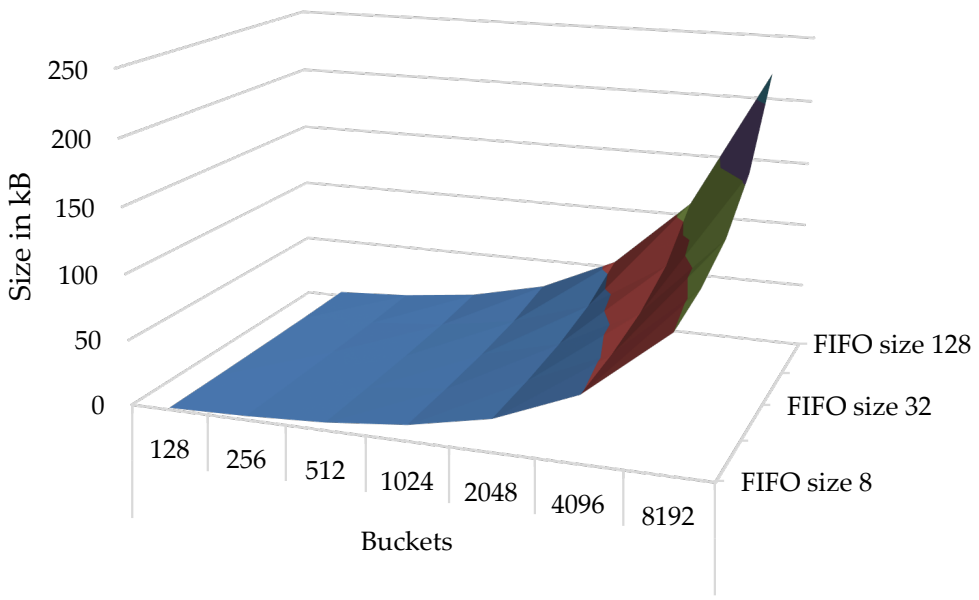

Figure 5.6: Storage requirements of BRC by bucket count and FIFO size in kB for stride detection

5.6.4 BRC Capacity Optimization conclusion

Training The best balance between overall storage size and performance is achieved by a configuration with 4096 buckets and a FIFO queue size of 16 (26 kB), reaching an average MPKI of 8.48, representing only a marginal increase of 2.12% compared to the uncapped baseline and a reduction of mispredictions by 10.97% relative to TAGE-SC-L.

Stride detection The best balance between overall storage size and performance is achieved by a configuration with 512 buckets and a FIFO queue size of 32 (6.5 kB), reaching an average MPKI of 9.48, representing a decrease of mispredictions by 0.81% compared to the uncapped baseline and a reduction of mispredictions by 2.78% relative to TAGE-SC-L.

5.7 HTP-Branch Analysis

To further evaluate the impact of the BRP, with training and stride detection, the HTP- branches are analyzed. For this evaluation, HTP- branches are defined as the branches (sorted from largest to smallest) that contribute to 90% of mispredictions

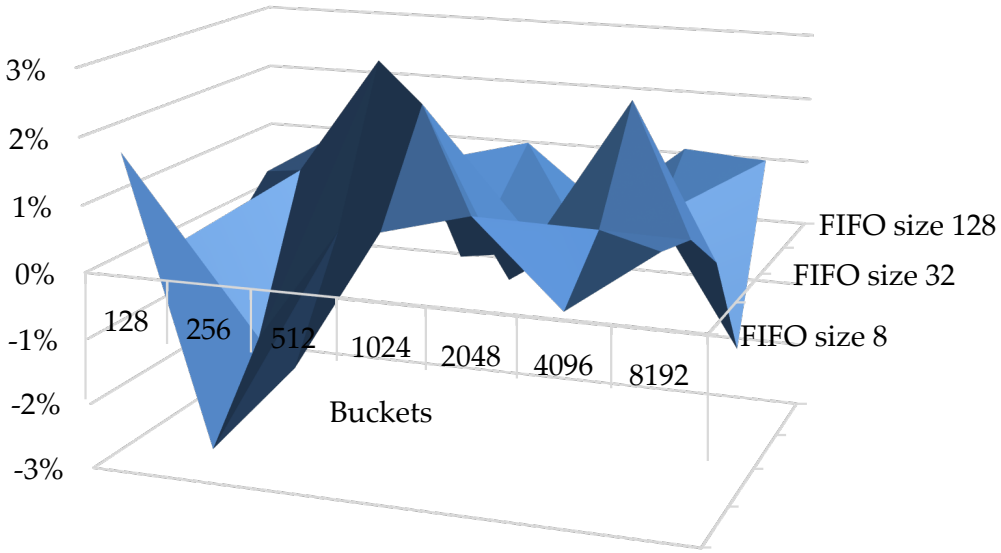

Figure 5.7: MPKI of recycling with stride detection relative to TAGE-SC-L in % (higher better)

under TAGE-SC-L. Rare branches with either 0 or 1 mispredictions are removed from the data set and are also not included in the branch count. TAGE-SC-L, is benchmarked against recycling with training and 4096 buckets, 16 bucket Size and stride detection with 512 buckets and bucket Size of 32. Table 5.24 displays the number of total- and HTP branches. Further, it displays the accuracy of both the training and stride detection mechanisms. The training mechanism yields a higher accuracy in detecting beneficial branches than the stride detection. Still, both mechanisms leave a large room for improvement, struggling heavily under workloads such as `505.mcf` and `compression`.

The mispredictions relative to TAGE-SC-L are displayed in Table 5.25. To evaluate the accuracy of the training and stride detection mechanism, the HTP-branches are evaluated with a perfect predictor choice. A perfect predictor choice is defined as follows: If a branch has fewer mispredictions with recycling under either training or stride detection than TAGE-SC-L, the BRP mispredictions are aggregated else, the mispredictions of the base-predictor TAGE-SC-L. The potential for both the training and stride detection eliminates an increase of mispredictions relative to TAGE-SC-L, highlighting that more refined training and stride detection mechanisms outperform TAGE-SC-L in all benchmarks by reducing mispredictions between 0,8% - 38.56% for training and 0.35%-20.48% for stride detection. The training mechanism delivers

Table 5.24: HTP-Branches with fewer mispredictions than TAGE-SC-L (64 kB) in % (higher better)

Benchmark	Branch Count	HTP Branch Count	Training	Stride Detection
505.mcf	104	10	30.00%	40.00%
523.xalancbmk	1 025	54	70.37%	62.96%
531.deepsjeng	889	206	83.98%	60.98%
541.leela	708	134	72.39%	45.64%
compression	129	13	23.08%	38.46%
dacapo-luindex	2 929	23	82.61%	39.13%
dacapo-lusearch	5 259	180	69.44%	50.00%
dacapo-spring	44 265	10 000	50.86%	45.64%
mediawiki	28 651	2 000	76.89%	56.03%
nodeapp	65 543	26 000	55.52%	52.42%

a higher potential for each benchmark, highlighting its superiority over the stride detection.

Table 5.25: Misprediction of HTP-branches relative to TAGE-SC-L (64 kB) with potential in % (higher better)

Benchmark	Training	Potential Training	Stride	Potential Stride
505.mcf	-5.51%	0.80%	-0.31%	0.63%
523.xalancbmk	30.32%	34.50%	3.70%	17.43%
531.deepsjeng	28.49%	29.89%	2.83%	9.23%
541.leela	11.05%	13.69%	0.15%	6.90%
compression	-7.22%	1.37%	-0.63%	0.35%
dacapo-luindex	27.90%	38.56%	-9.20%	5.76%
dacapo-lusearch	-5.60%	10.38%	0.44%	5.03%
dacapo-spring	12.78%	19.21%	0.15%	6.90%
mediawiki	20.43%	23.13%	0.77%	7.49%
nodeapp	22.18%	27.25%	13.81%	20.48%

6 Summary and Conclusion

This thesis investigated the concept of recycling executed branch outcomes as an extension to state-of-the-art branch predictors. The key idea is to reuse previously resolved false path branch results rather than discarding them after execution This provides an additional source of prediction information for HTP branches.

A BRP recycling executed outcomes was designed and implemented in the gem5 simulator, using TAGE-SC-L as the base predictor. The design aims to maintain correctness by recycling only executed outcomes and uses a bounded data structure (BRC) to limit storage overhead. Two optional control mechanisms, per-branch training and a stride detection mechanism, were introduced to selectively enable recycling only when it benefits prediction accuracy.

The evaluation shows that recycling is highly effective for repetitive workloads and branches with stable behavior. In a controlled microbenchmark, mispredictions were reduced by over 40%. On a standard benchmark suite, the recycling mechanism with training achieves an average misprediction reduction of up to 12.8% relative to TAGE-SC-L. Importantly, these improvements can be achieved with modest storage requirements of 18 `kB`, as bucket count was identified as the primary factor influencing accuracy.

Overall, the results demonstrate that recycling executed branch outcomes is a viable and promising technique for improving branch prediction accuracy. This makes branch outcome recycling a valuable complementary mechanism for future branch predictor designs.

7 Future Work

7.1 Branch Recycling Cache (BRC)

The BRC could be further improved by introducing a global limit on the total number of stored recycled outcomes, rather than statically capping the number of buckets and entries per bucket. Such a design would allow storage to be dynamically reallocated among branches based on observed usefulness through a training mechanism. This could potentially improve accuracy in workloads where only a small number of branches benefit from recycling, enabling them to store more outcomes. Additionally, a more sophisticated replacement policy could be explored. Instead of purely frequency-based eviction, future designs could explore benefit-based approaches, such as misspredictions that consider the historical impact of a specific branch.

7.2 Training Mechanism

The current training mechanism relies on simple misprediction counts of both predictors within a fixed execution window. While effective in many cases, more advanced training strategies could further improve accuracy and robustness across diverse workloads. The limits of the current training mechanism are especially evident in the workload 505.mcf, with a 5.2% increase in mispredictions. The results show that better targeting of beneficial branches leads to fewer mispredictions across all benchmarks relative to TAGE-SC-L.

7.3 Resource Usage

While the BRC is relatively small, a detailed analysis of its energy consumption and physical hardware cost would be necessary for a real-world implementation. Future work could evaluate the energy efficiency of branch outcome recycling and compare its cost-benefit trade-offs with those of alternative prediction mechanisms.

7.4 Selection of Base Predictor

The impact of the BRP could be studied in combination with other branch predictors, indirect branch predictors, or value predictors.

7.5 Latency

In this work, the recycling mechanism is evaluated primarily from a functional perspective. Future work should model the access latency of the BRC and study its impact on the processor frontend.

8 AI Disclaimer

Several AI-assisted tools were used during the preparation of this thesis to support both writing and implementation efforts. GitHub Copilot was used to improve convenience during development for trivial tasks, such as formatting error messages or reading and writing to files. However, it did not directly affect code quality. For writing, tools such as Grammarly, DeepL, and LanguageTool were used to enhance clarity, consistency, and overall quality. ChatGPT was used to convert data from Excel tables into LaTeX format for improved visuals.

While these tools contributed to productivity and refinement, all content was thoroughly reviewed and validated by the author. I remain fully responsible for the correctness, coherence, and originality of the final work.

Abbreviations

BRC Branch Recycling Cache

BRP Branch Recycling Predictor

HTP Hard-to-Predict branches (branches that contribute to 90% of mispredictions)

LFU Least-Frequently Used

MPKI Mispredictions per Kilo Instructions

PC Program Counter

TAGE TAgged GEometric history length Predictor

TAGE-SC-L TAGE with Statistical Corrector and Loop Predictor

TUM Technical University of Munich

List of Figures

3.1 System overview of the proposed BRP 8
3.2 Example overview of the BRC for training 9

5.1 Methodology of Evaluation 15
5.2 Reduction of mispredictions relative to TAGE-SC-L (64 kB) for training intervals (higher better) 19
5.3 Reduction in mispredictions relative to TAGE-SC-L (64 kB) for stride intervals (higher better) 21
5.4 Storage requirements of BRC by bucket count and FIFO size in kB for training 29
5.5 MPKI of recycling with training mechanism relative to TAGE-SC-L in % (higher better) 30
5.6 Storage requirements of BRC by bucket count and FIFO size in kB for stride detection 33
5.7 MPKI of recycling with stride detection relative to TAGE-SC-L in % (higher better) 34

List of Tables

4.1 Storage requirements for a bucket with training and N entries in bits . . 13
4.2 Storage requirements for a bucket with stride detection and N entries in bits 13

5.1 System configuration of gem5 14
5.2 Microbenchmark results in MPKI (lower better) 16
5.3 Reduction in mispredictions relative to TAGE-SC-L (64 kB) in % (higher better) 16
5.4 Overview of benchmarks with MPKI of TAGE-SC-L (64 kB) 17
5.5 MPKI of Benchmark Suite for training interval (lower better) 18
5.6 Reduction of mispredictions relative to TAGE-SC-L (64 kB) for training intervals in % (higher better) 18
5.7 MPKI of benchmark suite for stride interval (lower better) 20
5.8 MPKI of benchmark suite for bucket count with training (lower better) 23
5.9 MPKI relative to an uncapped BRC for capped bucket count under training (lower better) 23
5.10 MPKI of benchmark suite for bucket count with stride detection (lower better) 24
5.11 MPKI relative to an uncapped BRC for capped bucket count under stride detection (lower better) 24
5.12 MPKI of benchmark suite for FIFO queue size with training (lower better) 25
5.13 MPKI relative to an uncapped BRC for capped FIFO queues under training (lower better) 26
5.14 MPKI of benchmark suite for FIFO queue size with stride detection (lower better) 26
5.15 MPKI relative to an uncapped BRC for capped FIFO queues under stride detection (lower better) 27
5.16 MPKI for training with capped cache (lower better) 28
5.17 MPKI reduction relative to training with uncapped cache in % (lower better) 28
5.18 Storage requirements of BRC for training in kB 28

5.19 MPKI reduction relative to TAGE-SC-L for recycling with training in % (higher better) 28
5.20 MPKI for stride detection with capped cache (lower better) 31
5.21 MPKI reduction relative to stride detection with uncapped cache in % (lower better) 31
5.22 Storage requirements of BRC for stride detection in kB 31
5.23 MPKI reduction relative to TAGE-SC-L for recycling with stride detection in % (higher better) 32
5.24 HTP-Branches with fewer mispredictions than TAGE-SC-L (64 kB) in % (higher better) 35
5.25 Misprediction of HTP-branches relative to TAGE-SC-L (64 kB) with potential in % (higher better) 35

Bibliography

[Bla+06] S. M. Blackburn, R. Garner, C. Hoffmann, A. M. Khang, K. S. McKinley, R. Bentzur, A. Diwan, D. Feinberg, D. Frampton, S. Z. Guyer, M. Hirzel, A. Hosking, M. Jump, H. Lee, J. E. B. Moss, A. Phansalkar, D. Stefanović, T. VanDrunen, D. von Dincklage, and B. Wiedermann. "The DaCapo benchmarks: java benchmarking development and analysis." In: *SIGPLAN Not.* 41.10 (Oct. 2006), pp. 169–190. ISSN: 0362-1340. DOI: 10.1145/1167515.1167488.

[Goo26] Google Fleetbench Team. *Fleetbench: A Benchmarking Suite for Google Workloads*. https://github.com/google/fleetbench. Open-source benchmark suite, accessed: 2026-02-01. 2026.

[Löb17] A. Löbel. *SPEC CPU2017 Benchmark 505.mcf_r: Minimum-Cost Flow*. https://www.spec.org/cpu2017/Docs/benchmarks/505.mcf_r.html. Combinatorial optimization benchmark derived from a minimum-cost flow solver for single-depot vehicle scheduling. Standard Performance Evaluation Corporation (SPEC), 2017.

[Low+20] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, A. Armejach, N. Asmussen, S. Bharadwaj, G. Black, G. Bloom, B. R. Bruce, D. R. Carvalho, J. Castrillón, L. Chen, N. Derumigny, S. Diestelhorst, W. Elsasser, M. Fariborz, A. F. Farahani, P. Fotouhi, R. Gambord, J. Gandhi, D. Gope, T. Grass, B. Hanindhito, A. Hansson, S. Haria, A. Harris, T. Hayes, A. Herrera, M. Horsnell, S. A. R. Jafri, R. Jagtap, H. Jang, R. Jeyapaul, T. M. Jones, M. Jung, S. Kannoth, H. Khaleghzadeh, Y. Kodama, T. Krishna, T. Marinelli, C. Menard, A. Mondelli, T. Mück, O. Naji, K. Nathella, H. Nguyen, N. Nikoleris, L. E. Olson, M. S. Orr, B. Pham, P. Prieto, T. Reddy, A. Roelke, M. Samani, A. Sandberg, J. Setoain, B. Shingarov, M. D. Sinclair, T. Ta, R. Thakur, G. Travaglini, M. Upton, N. Vaish, I. Vougioukas, Z. Wang, N. Wehn, C. Weis, D. A. Wood, H. Yoon, and É. F. Zulian. "The gem5 Simulator: Version 20.0+." In: *CoRR* abs/2007.03152 (2020). arXiv: 2007.03152.

[LT19] C.-K. Lin and S. J. Tarsa. "Branch Prediction Is Not A Solved Problem: Measurements, Opportunities, and Future Directions." In: *2019 IEEE Inter-*

national Symposium on Workload Characterization (IISWC). 2019, pp. 228–238. DOI: 10.1109/IISWC47752.2019.9042108.

[Pas17] G.-C. Pascutto. *541.leela_r SPEC CPU2017 Benchmark*. Go playing engine using Monte Carlo tree search and pattern evaluation; C++ integer benchmark. Standard Performance Evaluation Corporation (SPEC), 2017.

[PRL+19] A. Prokopec, A. Rosà, D. Leopoldseder, et al. *Renaissance Benchmark Suite Documentation*. https://renaissance.dev/docs. Accessed: 2026-02-1. 2019.

[Sch26] D. Schall. *gem5 Server Benchmark Suite*. https://github.com/dhschall/gem5-svr-bench/tree/main/benchmarks. GitHub repository, accessed: 2026-02-1. 2026.

[Sez16] A. Seznec. "TAGE-SC-L Branch Predictors Again." In: *5th JILP Workshop on Computer Architecture Competitions (JWAC-5): Championship Branch Prediction (CBP-5)*. Seoul, South Korea, June 2016.

[SM06] A. Seznec and P. Michaud. "A case for (partially) TAgged GEometric history length branch prediction." In: *Journal of Instruction-level Parallelism - JILP* 8 (Feb. 2006).

[SPE17a] SPEC CPU2017 Benchmark Suite. *523.xalancbmk_r SPEC CPU2017 Benchmark*. Xalan-C++ XML transformation benchmark from the SPEC CPU2017 suite; transforms XML documents via XSLT. Standard Performance Evaluation Corporation (SPEC), 2017.

[SPE17b] SPEC CPU2017 Benchmark Suite. *531.deepsjeng_r SPEC CPU2017 Benchmark*. Deep learning / game tree evaluation workload in the SPEC CPU2017 suite. Standard Performance Evaluation Corporation (SPEC), 2017.